

FlaskGuard

Custom Web Application Firewall

Technical Report – Part 2

Regex-Based Attack Detection, Structured Attack Logger,
& Docker Containerisation

Repository:	<code>github.com/echoenvoy/FlaskGuard</code>
Report Part:	2 of 5
Author:	Saaf Ayyoub
Key Files:	<code>security.py</code> , <code>attack_logger.py</code> , <code>Dockerfile</code>
Date:	June 2026

Contents

1	Introduction and System Overview	3
1.1	Core Capabilities	3
2	Configuration System	3
2.1	Design Goals	3
2.2	YAML Configuration Schema – <code>config.yml</code>	3
2.3	Environment Variable Overrides	4
2.4	Configuration Loading Implementation	4
3	Rate Limiting with flask-limiter	5
3.1	Motivation	5
3.2	flask-limiter Integration	5
3.3	Rate Limit Configuration Deep Dive	5
3.4	Storage Backend for Rate Limit State	6
3.5	Rate Limiting and the Admin Dashboard	6
4	Admin Authentication – HTTP Basic Auth	6
4.1	Design Decision	6
5	Regex-Based Attack Detection – <code>security.py</code>	6
5.1	Why Regex?	6
5.2	Attack Categories Detected	7
5.3	Detection Logic – Inspected Input Sources	7
5.4	Regex Pattern Library	7
5.4.1	SQL Injection Patterns	7
5.4.2	XSS Patterns	7
5.4.3	Path Traversal Patterns	8
5.4.4	Command Injection Patterns	8
5.4.5	File Injection Patterns	8
5.5	Detection Function Return Value	8
5.6	Test Payloads	8
6	Attack Logger – <code>attack_logger.py</code>	9
6.1	Design Goals	9
6.2	Dual-Output Architecture	9
6.3	JSON Event Schema	9
6.4	Plaintext Security Log Format	9
7	Admin Security Dashboard	9
7.1	Overview and Design Philosophy	10
7.2	Dashboard Features	10
7.3	Backend API Endpoints	10
7.4	Manual Ban/Unban via curl	10
8	Docker Containerisation	11
8.1	Rationale for Containerisation	11
8.2	Dockerfile Analysis	11

8.3	docker-compose.yml Analysis	11
8.4	Network Architecture	11
8.5	Environment Variable Injection	12
8.6	Protecting Any Real Backend	12
9	Complete Request Processing Flow	12
9.1	Attack Event Lifecycle Summary	13
10	Conclusion	13

Introduction and System Overview

FlaskGuard is a custom Web Application Firewall designed to protect HTTP backends by inspecting incoming requests for malicious patterns, enforcing rate limits, and providing operators with visibility through an administrative dashboard. The system follows a modular architecture where each component addresses a specific security concern.

Core Capabilities

- **Configuration System:** YAML-based configuration with environment variable overrides following Twelve-Factor App methodology.
- **Rate Limiting:** IP-based request throttling using `flask-limiter` with tiered policies (e.g., 200 per day, 100 per hour).
- **Attack Detection:** Dual-layer detection using regex patterns and a machine learning classifier (ML covered in subsequent reports).
- **Structured Logging:** Dual-output logging to JSON (machine-readable) and plain-text (human-readable) formats.
- **Admin Dashboard:** Real-time visualization of attacks, blacklist management, and ML model status.
- **Docker Deployment:** Containerised deployment with network isolation and environment-based configuration.

Configuration System

Design Goals

The configuration subsystem was designed with three goals:

- **Human-readable defaults:** Operators can understand and edit the configuration without reading source code.
- **Environment-variable overrides:** Container deployments can inject secrets (admin credentials, backend URL) without baking them into a file.
- **Comprehensive coverage:** Every significant runtime behavior (rate limits, file paths, ban thresholds, ML confidence) is configurable without code changes.

YAML Configuration Schema – `config.yml`

The complete annotated configuration schema is reproduced and explained below:

Listing 1: `config.yml` – full schema with annotations

```
1 # Target backend that FlaskGuard protects
2 backend_url: "http://backend:8080"
3
4 # Port FlaskGuard listens on (overridden by PORT env var)
5 port: 5000
6
7 # Flask debug mode -- always false in production
8 debug: false
9
10 # Admin dashboard authentication
11 admin_auth:
12   enabled: true
```

```

13  username: "admin"      # Override via ADMIN_USERNAME env var
14  password: "admin123"   # Override via ADMIN_PASSWORD env var
15
16  # File paths for persistent storage
17  storage:
18    log_dir: "flaskguard/logs"
19    attacks_file: "flaskguard/logs/attacks.json"
20    security_log_file: "flaskguard/logs/security.log"
21    blacklist_file: "flaskguard/logs/blacklist.json"
22    classifier_model_file: "flaskguard/models/classifier.pkl"
23    classifier_meta_file: "flaskguard/models/model_meta.json"
24    feedback_file: "flaskguard/logs/feedback.json"
25
26  # Rate limiting configuration (flask-limiter)
27  rate_limiting:
28    enabled: true
29    default_limits:
30      - "200 per day"
31      - "100 per hour"
32
33  # Reverse proxy settings
34  proxy:
35    timeout_seconds: 10    # Max wait for backend response
36    verify_tls: true       # Validate backend TLS certificate
37
38  # IP-based automatic banning
39  ip_blacklist:
40    enabled: true
41    threshold_strikes: 5    # Attacks before auto-ban
42    ban_duration_minutes: 60
43
44  # ML classifier settings
45  ml_classifier:
46    enabled: true
47    confidence_threshold: 0.70 # Min probability to flag as attack

```

Environment Variable Overrides

Four environment variables are supported, mapping to the most operationally sensitive configuration values:

Variable	YAML Key Override	Purpose
BACKEND_URL	backend_url	Target backend endpoint
PORT	port	FlaskGuard listening port
ADMIN_USERNAME	admin_auth.username	Dashboard username
ADMIN_PASSWORD	admin_auth.password	Dashboard password

This follows the Twelve-Factor App methodology [?], keeping credentials out of version control while still providing reasonable defaults for development.

Configuration Loading Implementation

The configuration loader follows a straightforward but robust pattern:

1. Read and parse `config.yml` using PyYAML.

2. For each environment variable that has a value, replace the corresponding YAML key in the loaded dictionary.
3. Return the merged configuration as a Python dictionary accessible throughout the application.

Storage paths in the configuration may be either absolute (`/var/log/flaskguard/`) or relative to the project root, allowing both direct-run and Docker deployments to specify appropriate paths.

Rate Limiting with flask-limiter

Motivation

Rate limiting is a foundational defence against several attack categories:

- **Brute-force attacks:** Credential stuffing and password spraying require sending many requests quickly.
- **Denial-of-Service:** An unthrottled endpoint can be overwhelmed by a single poorly-behaved or malicious client.
- **Automated scanning:** Vulnerability scanners probe every URL on a target; rate limiting significantly slows this process.
- **Scraping:** Competitors or scrapers extracting large volumes of data are slowed.

flask-limiter Integration

FlaskGuard uses the `flask-limiter` library, which wraps Flask's request lifecycle to enforce request rate limits per IP address:

Listing 2: flask-limiter setup in app.py

```

1 from flask_limiter import Limiter
2 from flask_limiter.util import get_remote_address
3
4 limiter = Limiter(
5     app,
6     key_func=get_remote_address,
7     default_limits=config["rate_limiting"]["default_limits"]
8 )

```

The `key_func=get_remote_address` parameter means each unique IP address has its own counter. When a limit is exceeded, `flask-limiter` automatically returns an HTTP 429 Too Many Requests response.

Rate Limit Configuration Deep Dive

The `default_limits` field accepts a list of string-encoded limit specifications. Multiple limits create a tiered policy:

Limit	Period	Effect
200	day	No IP may send more than 200 requests in a 24-hour window
100	hour	No IP may send more than 100 requests in any 60-minute window

Storage Backend for Rate Limit State

By default, `flask-limiter` uses an in-memory storage backend. For production deployments, Redis is supported:

Listing 3: Redis backend configuration

```
1 from flask_limiter.storage import RedisStorage
2
3 limiter = Limiter(
4     app,
5     key_func=get_remote_address,
6     default_limits=["200 per day", "100 per hour"],
7     storage_uri="redis://localhost:6379"
8 )
```

Production Consideration: The default in-memory storage is not suitable for production deployments behind a load balancer. Migrate to Redis or another shared storage backend before exposing FlaskGuard to real traffic at scale.

Rate Limiting and the Admin Dashboard

The `/admin` and `/admin/api/*` routes are exempted from the default rate limits, as the admin dashboard is accessed by a single operator and should not be throttled during an active security incident review.

Admin Authentication – HTTP Basic Auth

Design Decision

The `/admin` namespace is protected by HTTP Basic Authentication, balancing simplicity with adequate security:

- **Simplicity:** No session management, cookie handling, or login page HTML is required.
- **HTTPS dependency:** Basic Auth transmits credentials base64-encoded; it is only safe with HTTPS/TLS.
- **Configuration-driven credentials:** Username and password are read from `config.yml` / environment variables.

Regex-Based Attack Detection – `security.py`

Why Regex?

Regular expressions are the most widely deployed technique in commercial WAF rule engines. Reasons include:

- **Deterministic:** A regex either matches or it does not.
- **Interpretable:** A security analyst can read a pattern and understand what it matches.

- **Fast:** Compiled regex executes in microseconds per input.
- **Auditable:** Each rule corresponds to a known attack category.

Attack Categories Detected

security.py provides detection for five OWASP Top 10-aligned attack categories:

Category	Description
SQL Injection (SQLi)	Attempts to inject SQL keywords and syntax into query parameters: ' OR 1=1 -, UNION SELECT.
Cross-Site Scripting (XSS)	Injection of client-side scripts: <script>, onerror=, javascript:.
Path Traversal	Use of ../ sequences to read files outside the intended directory.
Command Injection	Injection of shell metacharacters (;, , &&) or OS commands.
File Injection	PHP stream wrappers (php://input) or remote file inclusion patterns.

Detection Logic – Inspected Input Sources

FlaskGuard inspects all of the following input sources:

Input Source	Flask Accessor
URL query string parameters	request.args
POST form fields	request.form
Raw request body	request.data
HTTP headers	Selected dangerous headers
URL path	request.path

Regex Pattern Library

SQL Injection Patterns

Listing 4: Representative SQLi detection patterns

```

1 SQL_PATTERNS = [
2     r"(\bOR\b|\bAND\b)\s+[\d'\"]",           # OR 1=1, AND 'a'='a'
3     r"--\s*$",                               # SQL comment
4     r";\s*(DROP|INSERT|UPDATE|DELETE)\b",    # Stacked queries
5     r"\bUNION\b.\s+\bSELECT\b",              # UNION-based extraction
6 ]

```

XSS Patterns

Listing 5: Representative XSS detection patterns

```

1 XSS_PATTERNS = [
2     r"<script[\s>]",                          # <script> tag
3     r"javascript\s*:",                        # javascript: protocol

```



```

4     r"on\w+\s*=",          # event handlers
5     r"eval\s*\(",          # eval() calls
6 ]

```

Path Traversal Patterns

Listing 6: Path traversal detection patterns

```

1 PATH_TRAVERSAL_PATTERNS = [
2     r"\.\.\/",             # Unix traversal
3     r"\.\.\\",             # Windows traversal
4     r"%2e%2e[/\\]",        # URL-encoded traversal
5     r"/etc/passwd",         # Direct Unix target
6 ]

```

Command Injection Patterns

Listing 7: Command injection detection patterns

```

1 CMD_INJECTION_PATTERNS = [
2     r";\s*(ls|cat|whoami|id|uname|pwd|wget|curl)\b",
3     r"\\s*(ls|cat|whoami|id|nc|bash|sh)\b",
4     r"'`[\^']+',"          # Backtick substitution
5 ]

```

File Injection Patterns

Listing 8: File injection detection patterns

```

1 FILE_INJECTION_PATTERNS = [
2     r"php://",              # PHP stream wrappers
3     r"file://",             # File protocol
4     r"https?:\/\/[^\+\/][^\+\/]+\.(php|asp|jsp|sh)", # Remote file inclusion
5 ]

```

Detection Function Return Value

The `detect_attack()` function returns either `None` or a dictionary:

Listing 9: Attack detection result structure

```

1 {
2     "attack_type": "SQL_INJECTION",
3     "pattern": r"\bUNION\b.\bSELECT\b",
4     "matched_input": "q",
5     "payload": "' UNION SELECT ...",
6     "method": "regex"
7 }

```

Test Payloads

Listing 10: Documented test payloads

```

1 # SQL Injection
2 curl "http://localhost/search?q=' OR 1=1 --"

```

```

3 # XSS
4 curl "http://localhost/search?q=<script>alert(1)</script>"
5 # Path Traversal
6 curl "http://localhost/search?q=../../etc/passwd"
7 # Command Injection
8 curl "http://localhost/search?q=; ls -la"
9 # File Injection
10 curl "http://localhost/search?q=php://input"
11 # Normal (should pass through)
12 curl "http://localhost/search?q=laptop"

```

Attack Logger – `attack_logger.py`

Design Goals

The attack logger serves multiple stakeholders: dashboard frontend, ML retraining pipeline, operator audit trail, and incident response.

Dual-Output Architecture

Output	Format	Purpose
<code>attacks.json</code>	JSON array	Machine-readable; consumed by dashboard and ML pipeline
<code>security.log</code>	Plaintext line	Human-readable; suitable for log shippers

JSON Event Schema

Listing 11: Attack event JSON schema

```

1 {
2   "timestamp": "2026-06-09T14:23:07.412Z",
3   "ip": "203.0.113.42",
4   "method": "GET",
5   "path": "/search",
6   "attack_type": "SQL_INJECTION",
7   "detection_method": "regex",
8   "payload": "' OR 1=1 --",
9   "matched_field": "q",
10  "confidence": null,
11  "user_agent": "curl/7.88.1"
12 }

```

Plaintext Security Log Format

Listing 12: `security.log` line format

```

1 2026-06-09 14:23:07 [ATTACK] IP=203.0.113.42 TYPE=SQL_INJECTION \
2   METHOD=regex PATH=/search PAYLOAD="' OR 1=1 --"

```

Admin Security Dashboard

Overview and Design Philosophy

The dashboard is a single-page HTML/CSS/JavaScript interface served at `/admin`, built without frontend frameworks to eliminate build toolchains and ensure transparency.

Dashboard Features

Feature	Description
Attack Statistics	Total attacks blocked, unique attacking IPs, attacks by category.
Recent Attack Log	Paginated table of recent attack events.
Detection Method Breakdown	Breakdown of regex vs. ML classifier triggers.
Top Attacking IPs	Ranked list of IPs by attack frequency.
Blacklist Management	View banned IPs, manual ban/unban controls.
ML Model Status	Model metadata and feature importances.
Simulate Attacks	Button to seed demo data for testing.

Backend API Endpoints

All API routes are under `/admin/api/` and protected by Basic Auth:

Endpoint	Method	Description
<code>/admin</code>	GET	Renders dashboard HTML
<code>/admin/api/stats</code>	GET	Aggregate attack statistics
<code>/admin/api/attacks</code>	GET	Recent attack log
<code>/admin/api/blacklist</code>	GET	Active IP bans
<code>/admin/api/ban</code>	POST	Manually ban an IP
<code>/admin/api/unban</code>	POST	Remove an IP ban
<code>/admin/api/simulate</code>	POST	Seed demo attack data
<code>/admin/api/ml/status</code>	GET	ML model metadata
<code>/admin/api/ml/retrain</code>	POST	Trigger model retrain
<code>/admin/api/ml/feedback</code>	POST	Submit classification feedback

Manual Ban/Unban via curl

Listing 13: Manual IP ban and unban via curl

```

1 curl -u admin:admin123 -X POST http://localhost/admin/api/ban \
2   -H "Content-Type: application/json" \
3   -d '{"ip":"203.0.113.42","reason":"manual block"}'
4
5 curl -u admin:admin123 -X POST http://localhost/admin/api/unban \
6   -H "Content-Type: application/json" \
7   -d '{"ip":"203.0.113.42"}'

```

Docker Containerisation

Rationale for Containerisation

Docker provides reproducibility, isolation, portability, and composability for FlaskGuard deployments.

Dockerfile Analysis

Listing 14: Dockerfile

```
1 FROM python:3.11-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY . .
6 EXPOSE 5000
7 CMD ["python", "-m", "flaskguard.app"]
```

docker-compose.yml Analysis

Listing 15: docker-compose.yml

```
1 version: "3.9"
2 services:
3   flaskguard:
4     build: .
5     ports:
6       - "80:5000"
7     environment:
8       - BACKEND_URL=http://backend:8080
9       - ADMIN_USERNAME=${ADMIN_USERNAME:-admin}
10      - ADMIN_PASSWORD=${ADMIN_PASSWORD:-admin123}
11     depends_on:
12       - backend
13     networks:
14       - waf_network
15
16   backend:
17     build: ./demo_backend
18     expose:
19       - "8080"
20     networks:
21       - waf_network
22
23 networks:
24   waf_network:
25     driver: bridge
```

Network Architecture

The backend service is deliberately not exposed to the host network, only accessible within the `waf_network`. This forces all traffic through FlaskGuard.

Service	Host Port	Internal Ports
flaskguard	80 (public)	5000 (container)
backend	None (not exposed)	8080 (container only)

Security Pattern: This network segmentation mirrors production WAF deployments where the origin server's IP is kept confidential.

Environment Variable Injection

Listing 16: Injecting credentials at runtime

```

1 # Linux/macOS
2 ADMIN_USERNAME="prod-admin" ADMIN_PASSWORD="Str0ng!P@ss" \
3   docker compose up --build

```

Protecting Any Real Backend

Listing 17: Configuring a real-world backend

```

1 backend_url: "http://host.docker.internal:3000" # Local Node.js app
2 backend_url: "http://django-app:8000"         # Another Docker
   service
3 backend_url: "http://192.168.10.50:8080"       # Internal network
   server

```

Complete Request Processing Flow

With all components integrated, the full request processing flow for a malicious request is:

1. HTTP request arrives at Flask.
2. `@before_request`: Admin auth check passes (not an admin route).
3. `@before_request`: Rate limiter checks – within limits, continue.
4. `@before_request`: IP blacklist check – IP is not banned, continue.
5. `@before_request`: `detect_attack(request)` is called:
 - All request parameters are extracted and normalised.
 - Each regex pattern is applied in sequence.
 - First match returns an attack descriptor.
6. Attack is detected. `log_attack()` is called:
 - Event appended to `attacks.json`.
 - Line written to `security.log`.
7. `ip_manager.record_strike()` increments the source IP's strike count.
8. If strikes exceed threshold, IP is auto-banned (written to `blacklist.json`).

9. HTTP 403 response returned to client. Proxy route is never reached.
10. Dashboard (polling `/admin/api/*`) displays the new attack event.

Attack Event Lifecycle Summary

Stage	Component	Action
1	security.py	Detect and classify the attack
2	attack_logger.py	Write JSON event + log line
3	ip_manager.py	Record strike, auto-ban if threshold reached
4	app.py	Return HTTP 403 to client
5	dashboard	Display event in real-time (via API polling)

Conclusion

FlaskGuard delivers a complete Web Application Firewall through the integration of several well-designed subsystems. The YAML-based configuration system with environment variable overrides enables flexible deployment across development and production environments. Rate limiting provides a first-line defence against brute-force and DoS attacks. The regex-based detection engine covers the five most common web attack categories with deterministic, auditable patterns. The dual-output logging system creates both machine-readable and human-readable audit trails. The admin dashboard gives operators real-time visibility and control over blacklisting and ML model status. Finally, Docker containerisation provides reproducible, portable deployments with security-conscious network isolation.

Together, these components transform FlaskGuard from a simple application skeleton into a functional, detectable, and deployable Web Application Firewall suitable for protecting HTTP backends in a variety of operational contexts.